Trevor Streng

7/6/2025

Assignment 3

For this assignment, I worked on completing the code for a network that transfers styles of one image to another. More precisely, I am taking the content of one image and adding the styles of a different image to create a new image.

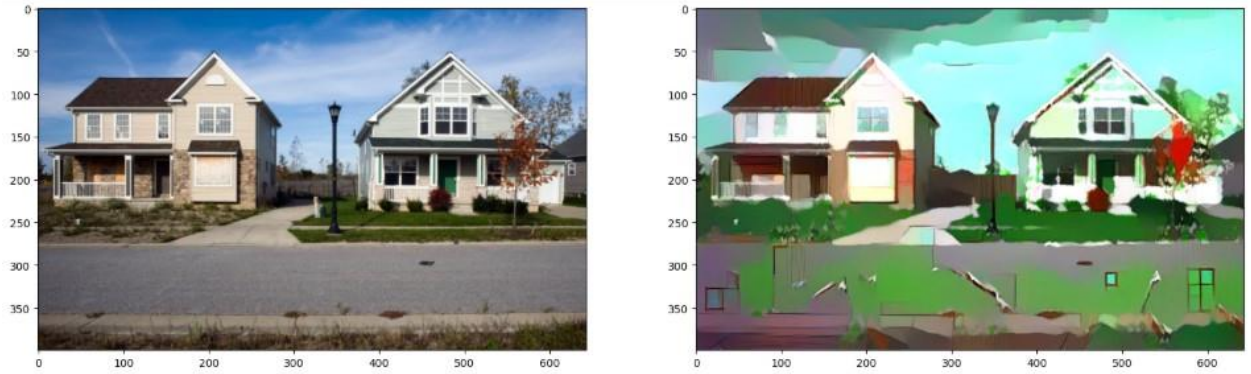
For part 1 of the assignment, I started by completing all the TODO portions of the code. First, I chose two images, one for style and one for content. After the images were loaded in, resized, and converted to numpy tensors, I completed mapping the layer names to the names found in the picture. I used slide 14 of the week 4.3 PowerPoint to help me visualize the different layers' names. Second, I worked on completing the Gram matrix function. Here, I started by getting the batch size, depth, height, and width of the tensor. Then, reshaped the tensor into a 2-dimensional tensor with one dimension being the depth, and the other being the height multiplied by the width. Using this feature tensor I calculated the Gram matrix using the torch.mm function with the features and the transpose of the features. After that, the content and style features were extracted, and each layer of the style feature was used to calculate the Gram matrix for each style layer. A target was also created as a copy of the content image. Third, it was time to calculate the content, style, and total losses. I started by getting the target features. Then, I could take the mean of the square of the target feature layer 'conv4_2' minus the content feature layer 'conv4_2' to get the content loss. Next, I moved on to calculating the style loss by plugging the target feature into the Gram matrix function and grabbing the layer for the needed style Gram calculated earlier. I then, subtracted these two values, squared them, and took the mean to get the style loss for that layer. I add this to the total style loss and repeat for each of the style weights that are needed. Finally, I can calculate the total loss by adding the content loss multiplied by the content weight(usually one) and the style loss multiplied by the style weight. This total loss is used to change the pixels of our target image.

For part 2 of the project, I started experimenting with the different hyperparameters. Before changing any of the parameters, I got pretty good results using two different groups of pictures. My next idea was to set each layer's weight to 1.0 while all others were at 0.1 to see what each layer was extracting. Using this method, I didn't see as much of a difference as I thought I would, even between conv1_1 and conv5_1. I did notice that small things seemed to be less distorted from the original content picture when the weight was given to later layers. I noticed that when using a picture of a cartoon house for the styles and a picture of two houses on a road for the content, it seemed to think the road was a background or something, and was adding random items from the style picture into the road. I was also getting a total loss that was

in the millions. I then turned down the beta weight and some of the earlier layers' weights. This seemed to help a little with the random items being generated in the middle of the road. After messing with the different layers' weights, I noticed that when adding more weight to the later layers, the color and some objects from the style image were more prominent. I then turned the beta weight all the way down to 10 and put the alpha weight up to 10. This dropped the loss a bit and kept a lot more of the contents image. It also helped reduce the amount of noise on the road. I decided I wanted it to have more of the style, so I went back to raising the beta weight to 1e4. Raising the learning rate seemed to be similar to raising the beta weight and made the style a little stronger. The hardest part of this project seems to be getting enough of the style through without having random objects/illusions appear.

In the end, I have decided my happy medium was having the alpha/content weight at 1, the beta/style weight at 1e4, conv1_1 at 0.5, conv2_1 at 0.6, conv3_1 at 0.8, conv4_1 at 0.6, conv5_1 at 0.4, and the learning rate at 0.005. My goal for making the middle layers' weights the heaviest was to see if I could extract less of the objects and produce fewer illusions while still getting a good amount of the style. Although this does include some illusions, it seems to apply a decent amount of style to the houses. My total loss for this was 15499. If there was too little loss, the picture tended to stay similar to the content image, while if the loss was too high, there seemed to be too many illusions.

Fig



1.

Conv1_1 = 1.0

All other layers were 0.1

Rest of hyperparameters were left alone.

Fig 2.

Conv5_1 = 1.0

All other layers were 0.1

Rest of hyperparameters were left alone.

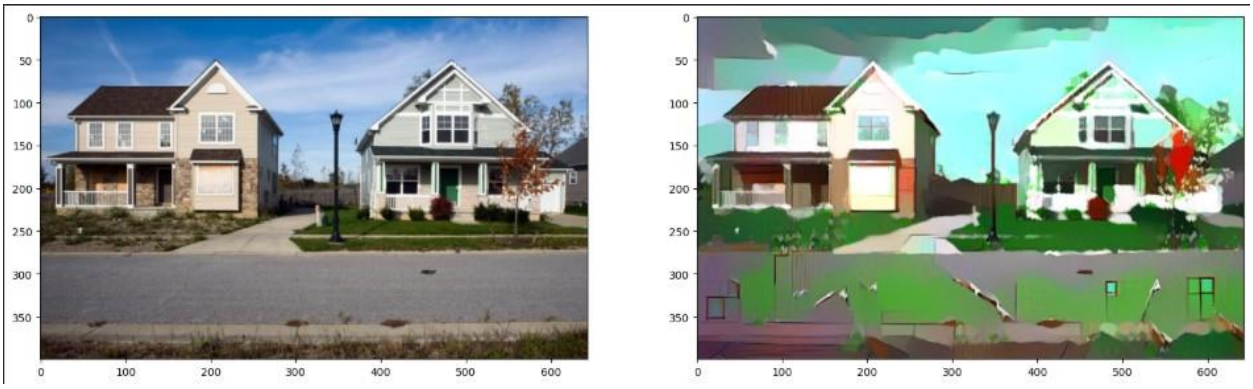


Fig 3.

Style_weights = { conv1_1 : 0.5,

conv2_1 : 0.6,

conv3_1 : 0.8,

conv4_1 : 0.6,

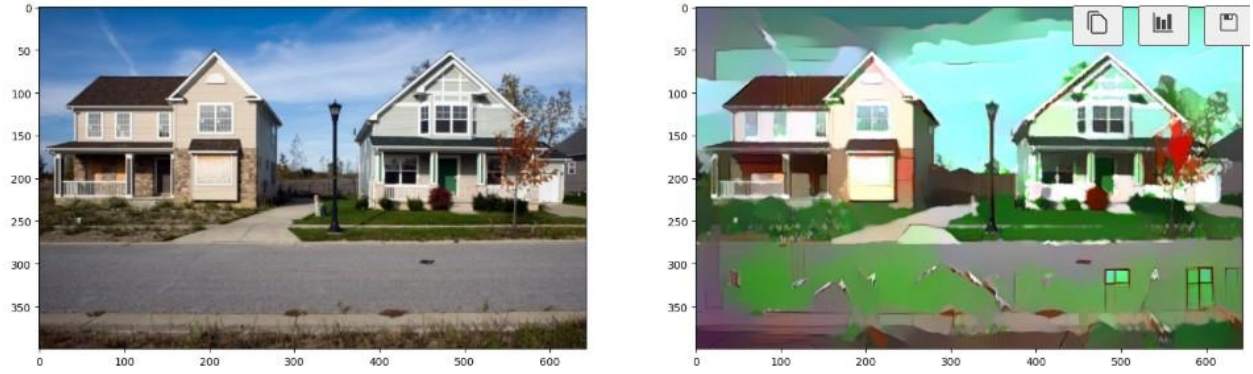
conv5_1 : 0.4 }

content_weight=1

style_weight=1e4

learning_rate = 0.005

steps = 3000



Works Cited

- https://quaternionidentity.com/neural-style-transfer-pytorch-english.html
    - Originally used to help me with Gram matrix, but also helped with calculating loss.

Final code snippets that may have been referenced above

Final weights

```
style_weights = {'conv1_1': 0.1,
                 'conv2_1': 0.4,
                 'conv3_1': 0.6,
                 'conv4_1': 0.8,
                 'conv5_1': 0.9}

# you may choose to leave these as is
content_weight = 1  # alpha
style_weight = 1e4  # beta
```